UniCore: Centralized Campus Operating Platform & High-Concurrency Transaction Layer

Project Proposal

Ankit Rath (1024030458) **Manan Kapoor** (1024030467) **Abhinav Kumar Singh** (10240304
Department of Computer Science & Engineering
Thapar Institute of Engineering & Technology, Patiala

August 17, 2026

Abstract

UniCore is a centralized campus operating platform engineered to unify student academic records, residential hostel allotment, library asset circulation, examination processing, and administrative workflows under a single system of record. Decomposed into Boyce-Codd Normal Form (BCNF) across 8 domain schemas and 35+ tables, UniCore guarantees 100% ACID transaction safety during peak concurrent rushes via explicit row-level locking (`SELECT FOR UPDATE`), automated PL/SQL triggers, and immutable JSON audit ledgers, targeting a Time-To-Acknowledgement (TTA) of ≤ 2 hours across 30,000+ active students.

1 Elevator Pitch

- **The Gap:** Modern universities run academics, hostels, library, and exams as disconnected system silos, causing severe data duplication, drifting records, room double-booking race conditions during peak rushes, and zero forensic auditability.
- **The Solution:** UniCore provides a centralized operating platform (8 BCNF schemas, 35+ tables) governed by atomic row locks (`SELECT FOR UPDATE`), automated PL/SQL triggers, and immutable JSON audit ledgers.
- **The Impact:** Eliminates administrative overhead, guarantees 100% ACID transaction safety during enrollment rushes, and achieves a target Time-To-Acknowledgement (TTA) ≤ 2 hours across 30,000+ students.

2 SMART Project Objectives

1. **Primary Objective:** Design, implement, and benchmark a unified BCNF operating platform for 30,000+ students that eliminates data redundancy and guarantees ACID transaction safety during peak concurrent rushes.
2. **Sub-Goal 1 (BCNF Normalization):** Decompose 35+ relational tables across 8 domain schemas strictly into Boyce-Codd Normal Form.
3. **Sub-Goal 2 (Atomic Concurrency Control):** Enforce row-level locking (`SELECT FOR UPDATE`) within stored procedures (`hostel_allot()`) to guarantee zero double-booking under 10,000+ concurrent requests.
4. **Sub-Goal 3 (Automated Triggers):** Implement PL/SQL `BEFORE INSERT` triggers blocking invalid operations (unpaid fines > Rs. 500) and `AFTER` triggers emitting JSON state diffs to an immutable `audit_logs` table.

5. **Sub-Goal 4 (Concurrency Benchmarking):** Evaluate transaction throughput (TPS) and latency percentiles (p95/p99) under high-concurrency loads using benchmarking tools.

3 System Architecture & Concurrency Controls

Scenario	Isolation Level	Mechanism	Enforced Guarantee
Hostel Bed Allotment	`SERIALIZABLE`	`SELECT FOR UPDATE`	Zero double-booking
Exam Seat Registration	`READ COMMITTED`	`pg_advisory_xact_lock()`	Seat count $\leq$ Capacity
Library Book Issue	`READ COMMITTED`	Decrement `total_copies`	Available count ≥ 0
Grade Record Updates	`READ COMMITTED`	`UPSERT`	Updates row without duplication

Table 1: UniCore Transaction Concurrency Matrix

4 UML and Data Flow Modeling

The architecture integrates UML Use Case, Class, Sequence, Component, and State Machine diagrams along with 3 levels of Data Flow Diagrams (DFD Level 0 Context, DFD Level 1 System Flow, DFD Level 2 Detailed Process Flow) to model end-to-end operational execution and forensic audit ledgers.

5 Timeline & Milestones

Weeks	Technical Task Focus	Key Deliverable
Weeks 1–3	Domain Requirements & ER Modeling	8-Domain ER Schematic & Proposal Approval
Weeks 4–6	BCNF Normalization & PostgreSQL DDL	35+ BCNF Tables Created
Weeks 7–8	PL/SQL Triggers & Row Locks	`hostel_allot()`, `fine_block_trigger`
Weeks 9–10	REST API Integration & Benchmarking	Concurrency Metrics Suite
Weeks 11–12	End-to-End Evaluation & Testing	Final Master Report & Staging Build

Table 2: Week-by-Week Development Schedule

6 Resources & Budget

- **Engineering Team:** Ankit Rath, Manan Kapoor, Abhinav Kumar Singh.
- **Budget: No external funding required.** All development executes on existing university laboratory infrastructure.